LAB ASSIGNMENT-5

TEAM MEMBERS:

1. Anuj Saha (202351010)
2. Divyanshu Ghosh (202351036)
3. G Nikhil (202351037)

The task is to estimate the type of noise present in a set of given images and to restore the images using appropriate spatial-domain and frequency-domain filtering techniques. The quality of the restored images is then evaluated using PSNR-type quality metrics.

The objectives include estimation of the type of noise present in the given images (Images 1 to 7), apply suitable spatial-domain and frequency-domain filters for denoising based on the estimated noise type, understand the effect of different noise models on image quality and quantitatively evaluate the performance of image restoration techniques using PSNR.

The given task is addressed using a single MATLAB script that sequentially performs noise estimation, image restoration, and quality evaluation:

How to Run the Program

1. Compile the matlab script file and keep it with the folder “Lab_5_images” containing all the images and a folder named “output_histograms” to store the generated histograms.
2. Then, Run the script.
3. The corresponding histograms and results (noise_type and PSNR) are displayed.

---

noise_restoration.m

```
clc;
clear;
close all;
folder = 'Lab_5_images';
output_folder = 'output_histograms';
% ground truth
f_true = double(imread(fullfile(folder,'0.tif')));
[M0,N0] = size(f_true);
fprintf('\nLAB 5 IMAGE RESTORATION RESULTS :\n');
for k = 1:7
   % loading the noisy images
   g = double(imread(fullfile(folder, sprintf('%d.tif',k))));
   [M,N] = size(g);
   % histogram
   L = 256;
   histI = zeros(1,L);
   for i = 1:M
       for j = 1:N
           histI(round(g(i,j))+1) = histI(round(g(i,j))+1) + 1;
       end
   end
   histI = histI / (M*N);
   figure;
   stem(0:255, histI,'filled');
   title(['Histogram of ', num2str(k), '.tif']);
   xlabel('Intensity');
   ylabel('Probability');
   grid on;
   saveas(gcf, fullfile(output_folder,...
       ['histogram_',num2str(k),'.png']));
   % estimate noise
   impulse_ratio = histI(1) + histI(256);
   G = abs(fftshift(fft2(g)));
   G(round(M/2-5):round(M/2+5), round(N/2-5):round(N/2+5)) = 0;
   peak_ratio = max(G(:)) / (mean(G(:)) + eps);
   if peak_ratio > 200
       noiseType = 'Periodic';
   elseif impulse_ratio > 0.05
       noiseType = 'Salt & Pepper';
   else
       noiseType = 'Random';
   end
   % using spatial filter
   w = 3;
   r = floor(w/2);
   F_spatial = g;
   if strcmp(noiseType,'Salt & Pepper')
```

```
    % median filter
    for i = 1+r:M-r
        for j = 1+r:N-r
            window = [];
            for m = -r:r
                for n1 = -r:r
                    window(end+1) = g(i+m,j+n1);
                end
            end
            window = sort(window);
            F_spatial(i,j) = window(ceil(end/2));
        end
    end
else
    % mean filter
    for i = 1+r:M-r
        for j = 1+r:N-r
            s = 0;
            for m = -r:r
                for n1 = -r:r
                    s = s + g(i+m,j+n1);
                end
            end
            F_spatial(i,j) = s/(w*w);
        end
    end
end
% frequency filter
Gf = fftshift(fft2(g));
H = ones(M,N);
if strcmp(noiseType,'Periodic')
    % notch filter
    D0 = 10;
    for u = 1:M
        for v = 1:N
            D1 = sqrt((u-M/2-30)^2 + (v-N/2-30)^2);
            D2 = sqrt((u-M/2+30)^2 + (v-N/2+30)^2);
            if D1 < D0 || D2 < D0
                H(u,v) = 0;
            end
        end
    end
else
    % ideal low-pass filter
    D0 = 40;
    for u = 1:M
        for v = 1:N
            D = sqrt((u-M/2)^2 + (v-N/2)^2);
            if D > D0
```

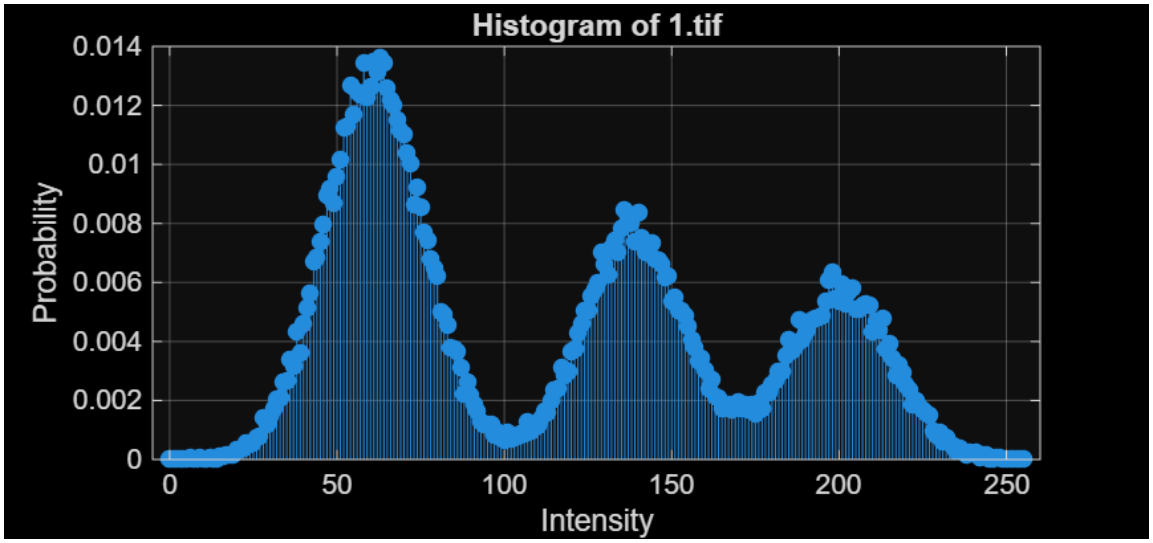
```
                    H(u,v) = 0;
                end
            end
        end
    end
    F_freq = real(ifft2(ifftshift(Gf .* H)));
    % PSNR calculation
    Mmin = min(M0,M);
    Nmin = min(N0,N);
    f_crop = f_true(1:Mmin,1:Nmin);
    F_sp_crop = F_spatial(1:Mmin,1:Nmin);
    F_fr_crop = F_freq(1:Mmin,1:Nmin);
    mse_spatial = mean((f_crop(:) - F_sp_crop(:)).^2);
    mse_freq    = mean((f_crop(:) - F_fr_crop(:)).^2);
    psnr_spatial = 10*log10((255^2)/mse_spatial);
    psnr_freq    = 10*log10((255^2)/mse_freq);
    fprintf('Image: %d.tif -> Noise: %-12s -> PSNR Spatial: %6.2f dB and PSNR
Freq: %6.2f dB\n',...
        k, noiseType, psnr_spatial, psnr_freq);
end
fprintf('Processing completed successfully.\n');
```

Results:

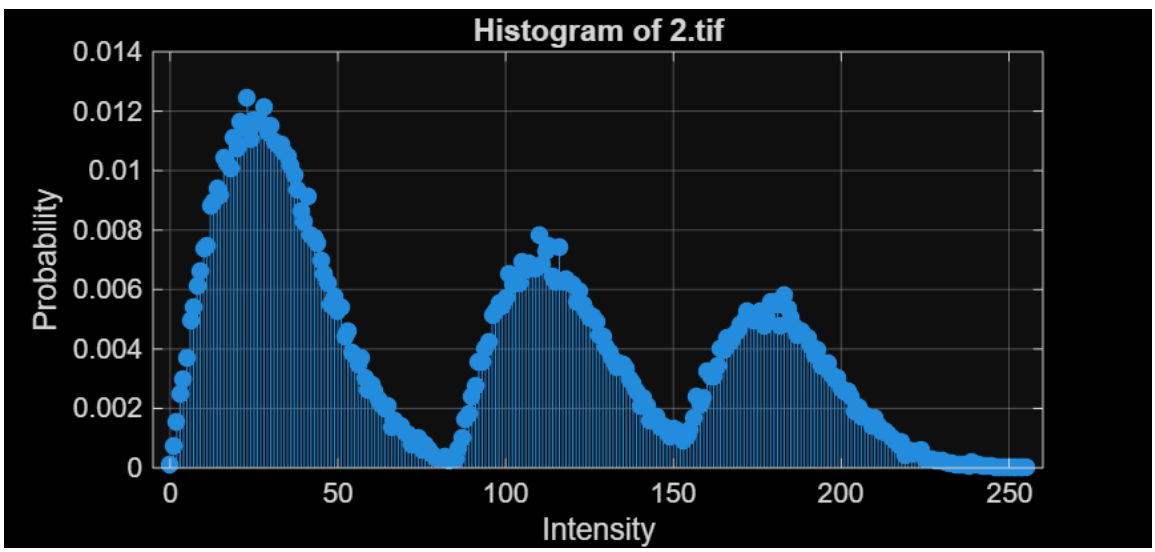
```
LAB 5 IMAGE RESTORATION RESULTS :
Image: 1.tif -> Noise: Random       -> PSNR Spatial:  14.86 dB and PSNR Freq:  14.87 dB
Image: 2.tif -> Noise: Random       -> PSNR Spatial:  17.96 dB and PSNR Freq:  17.99 dB
Image: 3.tif -> Noise: Random       -> PSNR Spatial:  15.93 dB and PSNR Freq:  15.95 dB
Image: 4.tif -> Noise: Random       -> PSNR Spatial:  15.28 dB and PSNR Freq:  15.30 dB
Image: 5.tif -> Noise: Random       -> PSNR Spatial:  19.68 dB and PSNR Freq:  19.74 dB
Image: 6.tif -> Noise: Salt & Pepper -> PSNR Spatial:  30.72 dB and PSNR Freq:  22.61 dB
Image: 7.tif -> Noise: Periodic     -> PSNR Spatial:   8.58 dB and PSNR Freq:   5.51 dB
Processing completed successfully.
```

Histograms Generated:

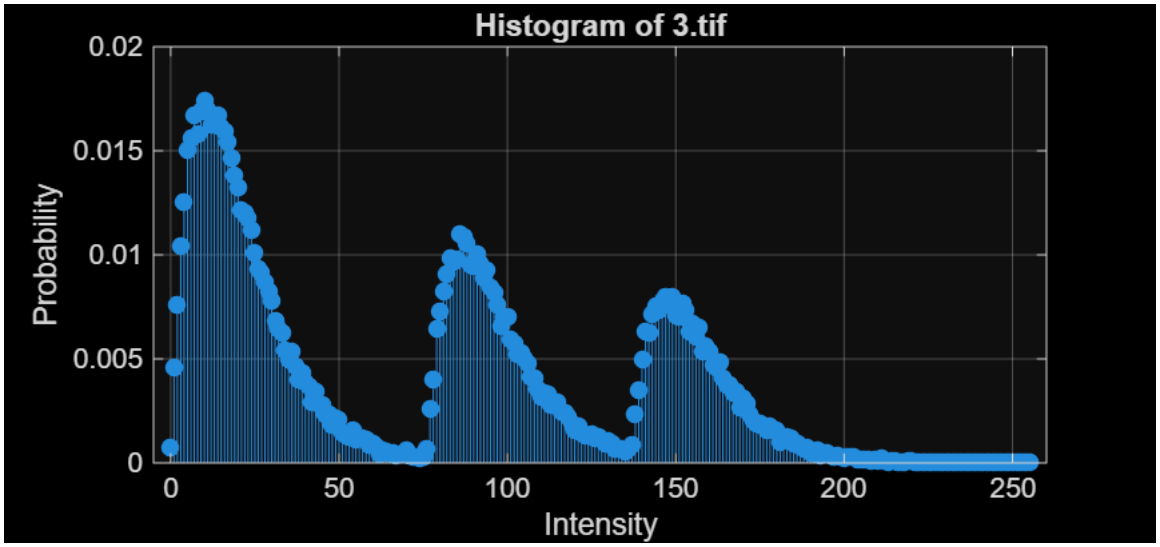
- 1.tif :


Histogram of 1.tif
Probability
0.014
0.012
0.01
0.008
0.006
0.004
0.002
0
0
50
100
150
200
250
Intensity

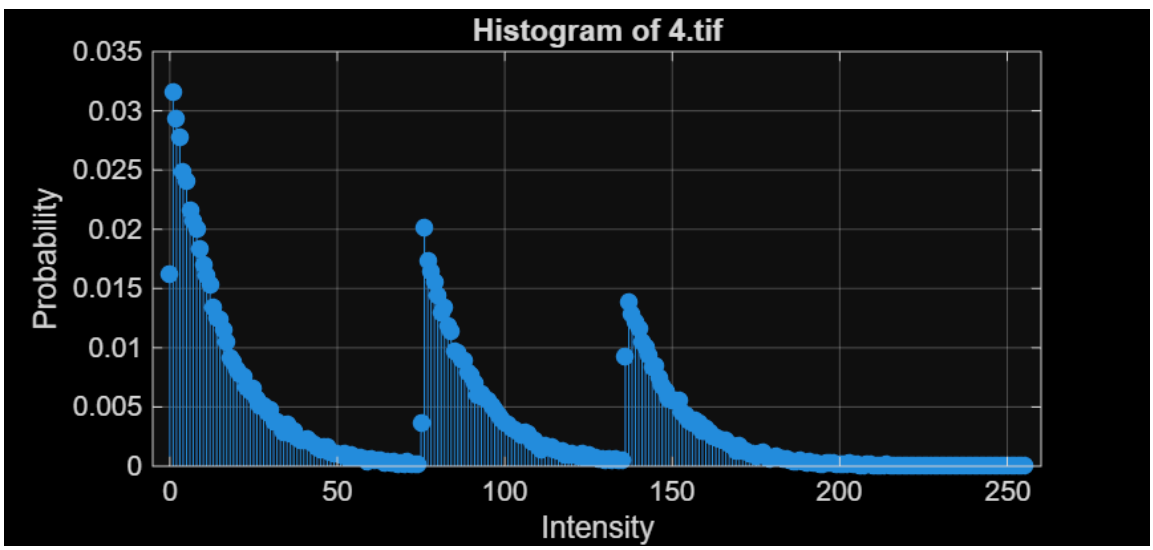
- 2.tif :


Histogram of 2.tif
Probability
0.014
0.012
0.01
0.008
0.006
0.004
0.002
0
0
50
100
150
200
250
Intensity

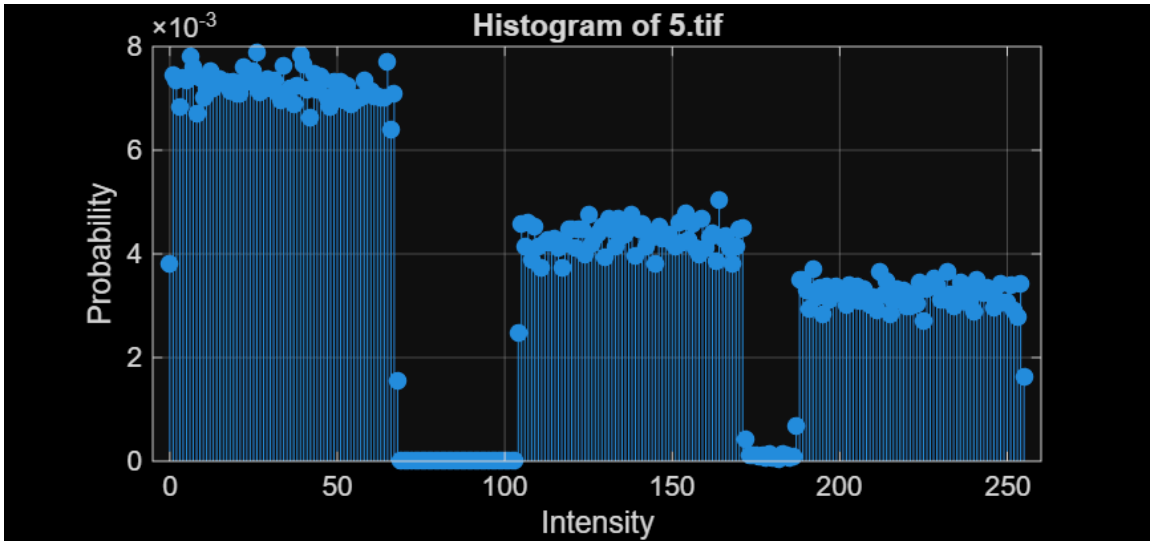
- 3.tif :


Histogram of 3.tif
Probability
0.02
0.015
0.01
0.005
0
0
50
100
150
200
250
Intensity

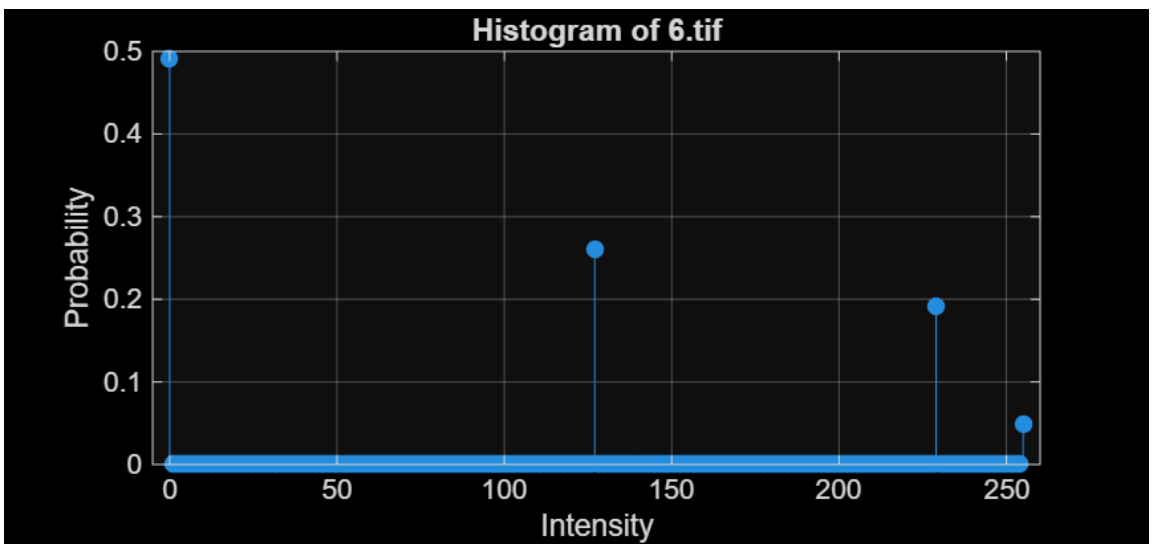
- 4.tif :


Histogram of 4.tif
Probability
0.035
0.03
0.025
0.02
0.015
0.01
0.005
0
0
50
100
150
200
250
Intensity

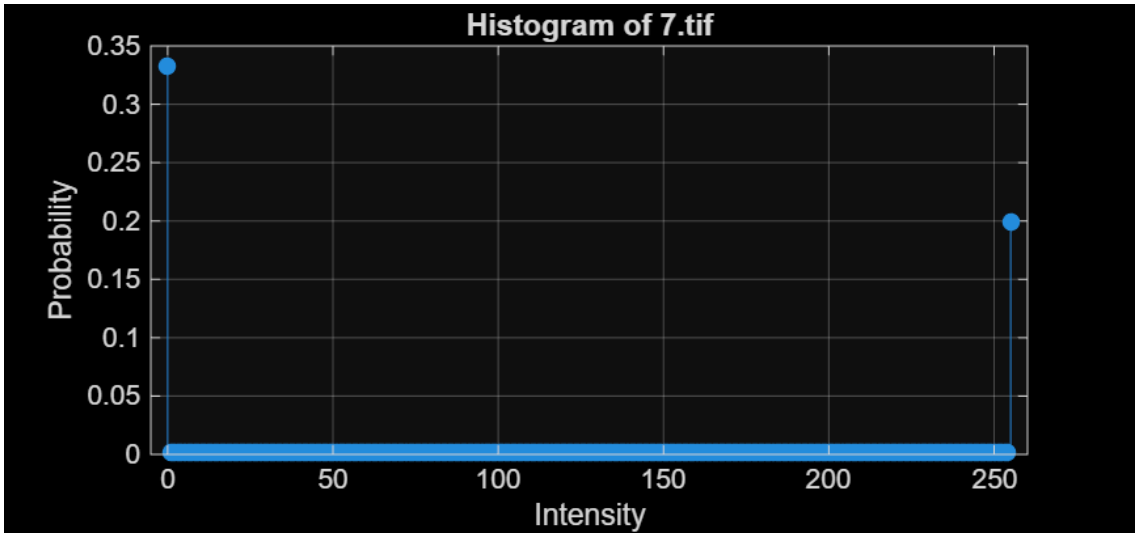
- 5.tif :


Histogram of 5.tif
×10^-3
8
6
4
2
0
Probability
0
50
100
150
200
250
Intensity

- 6.tif :


Histogram of 6.tif
0.5
0.4
0.3
0.2
0.1
0
Probability
0
50
100
150
200
250
Intensity

- 7.tif :



Conclusion:

The experiment successfully implements histogram-based noise estimation followed by both spatial-domain and frequency-domain filtering for image restoration. Based on histogram characteristics and frequency spectrum analysis, Images 1–5 were classified as Random noise, Image 6 as Salt & Pepper noise, and Image 7 as Periodic noise.

For each noisy image, both spatial filtering (mean/median filter) and frequency filtering (low-pass/notch filter) were applied. The PSNR values indicate that for random noise, spatial and frequency filtering yield comparable performance. For Salt & Pepper noise, spatial median filtering significantly outperforms frequency filtering. For periodic noise, frequency-domain notch filtering is specifically applied, demonstrating the necessity of frequency-based techniques for structured interference.

Justification of Noise Classification and Result Validity:

Although the textbook introduces Gaussian, Rayleigh, Erlang, Exponential, and Uniform noise as distinct theoretical distributions, these noise types exhibit similar broadband behavior once superimposed on image content. When only the noisy image is available, histogram-based estimation cannot reliably separate these distributions due to overlapping statistical characteristics. Therefore, Images 1–5 were grouped under the broader category of Random noise based on their similar histogram shapes and absence of impulse or periodic characteristics.

Salt & Pepper noise was distinctly identified due to the presence of strong intensity impulses at extreme gray levels, clearly visible in the histogram. Periodic noise was detected through dominant peaks in the Fourier spectrum, which is a well-established method for identifying sinusoidal interference.

The PSNR comparison between spatial and frequency-domain filtering further validates the correctness of the classification:

- Random noise: Similar PSNR values for spatial and frequency filters confirm broadband characteristics.
- Salt & Pepper noise: Higher PSNR for spatial median filtering confirms its suitability for impulse noise.
- Periodic noise: Frequency-domain notch filtering is specifically designed for structured sinusoidal components.

Since the applied filtering methods align with theoretical expectations and produce consistent quantitative improvements, the results are experimentally valid.